Project - High Level Design

On

Dockerized Construction Services

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Ansh Kumbhare	EN22CS301140
2.	Anshul Sen	EN22CS301158
3.	Anshul Yadav	EN22CS301159
4.	Armi Bhawsar	EN22CS301194
5.	Athrav Dubey	EN22CS301229

Group Name: Group06-D2

Project Number: DO-18

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Interfaces

4. Data Design
   4.1 Data Model
   4.2 Data Access Mechanism
   4.3 Data Retention Policies
   4.4 Data Migration

5. State and Session Management

6. Caching

7. Functional Requirements
   7.1 User Authentication & Authorization
   7.2 Infrastructure Configuration Management
   7.3 Provisioning Control C Monitoring
   7.4 Dockerfile (Application Containerization)
   7.5 GitHub Actions (Workflow Configuration)
   7.6 CI/CD Strategy (Continuous Integration C Deployment)

8. Non-Functional Requirements

9. References

10. GitHub

1. Introduction

This document presents the system design for the Dockerized Construction Service, developed as part of a DevOps-based project. The project demonstrates a complete containerization workflow using Docker along with CI/CD automation.

The application represents a Construction Planning Service integrated with GenAI capabilities, where intelligent modules assist in planning, scheduling, and resource estimation using AI-driven insights.

The DevOps pipeline ensures the application is:

- Consistently packaged
- Easily deployable
- Scalable across environments
- Secure and optimized

1.1. Scope of the document

This document includes:

- Docker-based containerization strategy
- Multi-stage Docker build optimization
- Docker Compose orchestration
- CI/CD pipeline using GitHub Actions / Jenkins
- Environment and dependency management
- Security and performance considerations
- API catalogue and service endpoints

1.2. Intended Audience

This document is intended for:

- DevOps Engineers
- Backend Developers
- AI Engineers
- Evaluators
- Future Maintainers

1.3. System overview

The system consists of two main repositories:

Repository 1 – Construction Service(Application)

Contains:

- Backend logic (construction planning)
- GenAI Modules (task suggestions, cost estimation)
- APIs and business logic

Repository 2 – DevOps Infrastructure

Contains:

- Docker file
- Docker compose
- CI/CD pipeline
- Environment configs

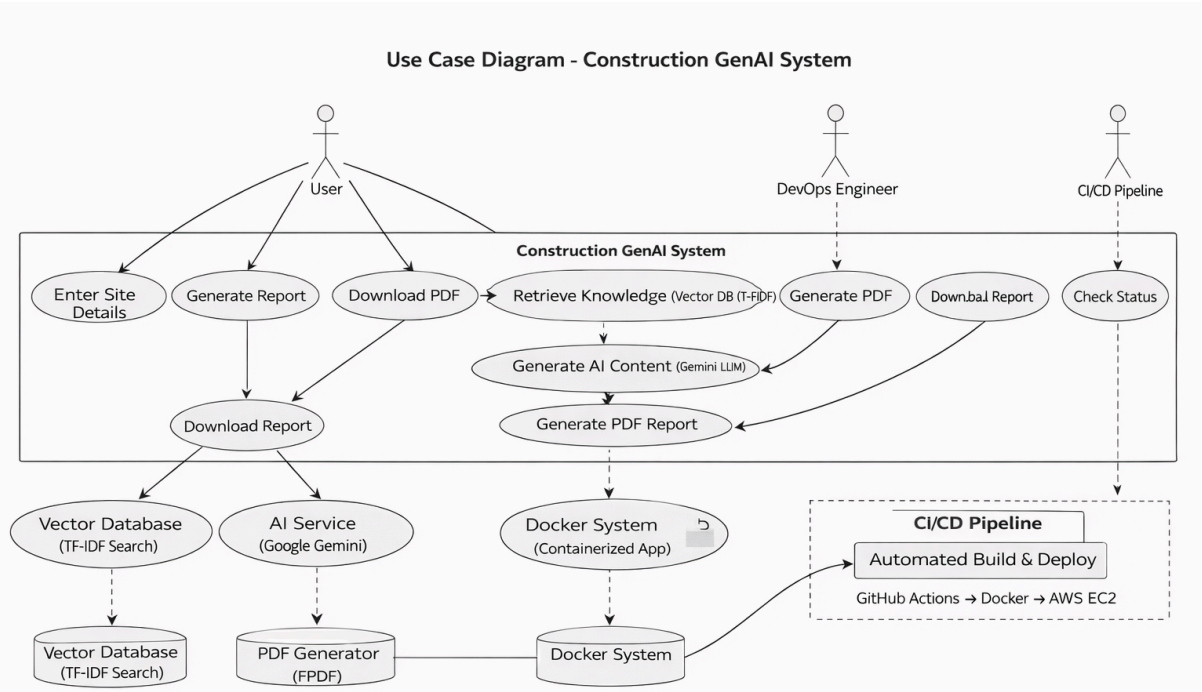
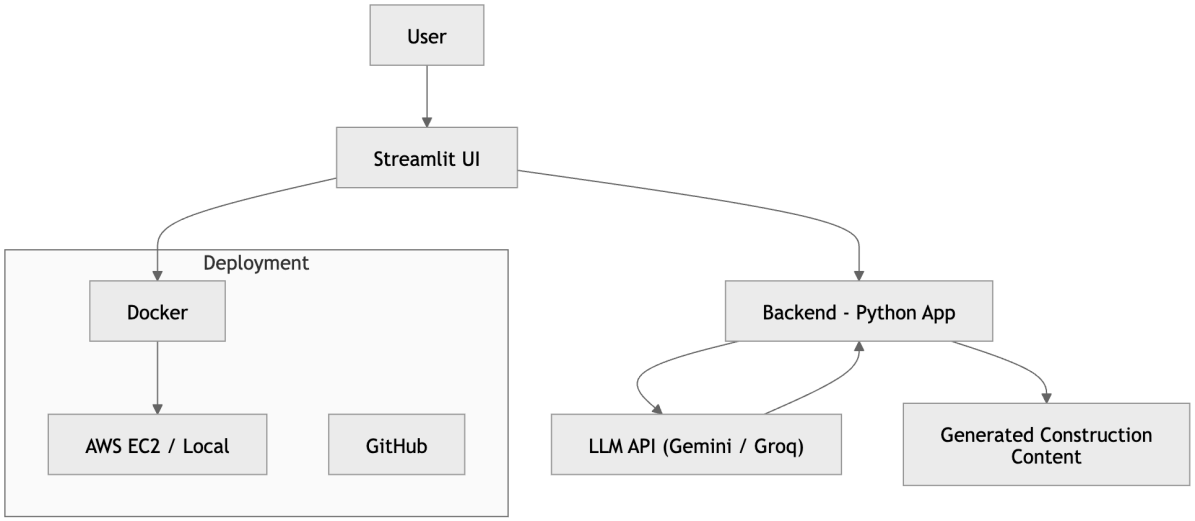


Figure 1: Use Case Diagram

High-Level Workflow

- Developer pushes code → GitHub
- CI/CD pipeline triggers
- Docker image is built using multi-stage build
- Image is stored in Docker Hub
- Docker Compose deploys services
- Application becomes accessible

2. System Design



2.1. Application Design

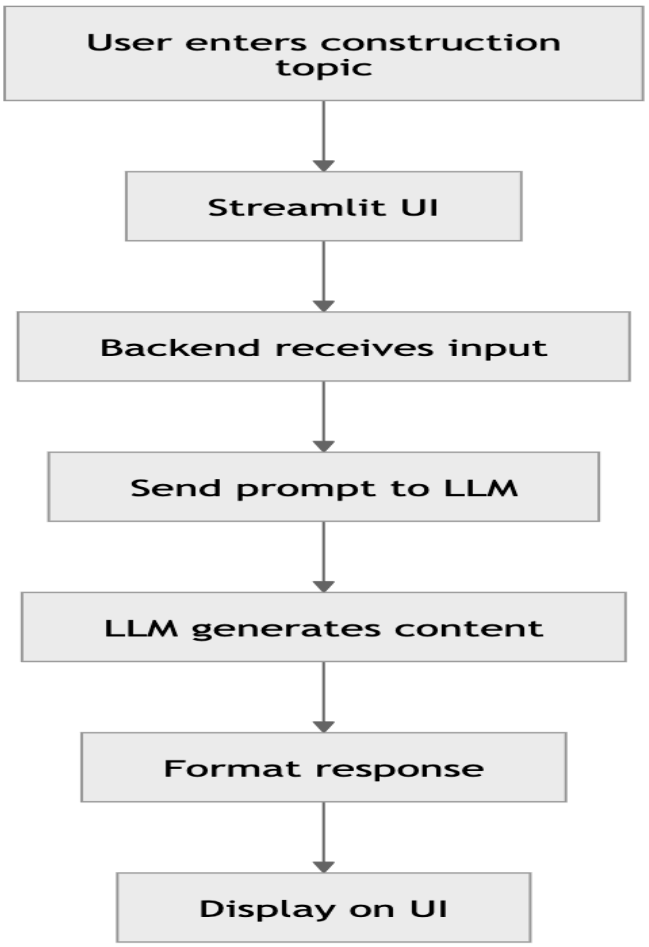
The **Construction GenAI Application** is designed as a modular system that integrates user input, AI processing, and report generation.

- The **frontend** is built using Streamlit, allowing users to enter construction site details and request reports.
- The **backend logic** processes user input, applies prompt engineering, and interacts with the AI model.
- A **vector database (TF-IDF)** is used to retrieve relevant documents from the knowledge base to enhance accuracy (RAG approach).
- The system integrates with **Google Gemini** to generate structured and context-aware construction reports.
- The generated content is converted into a **PDF report using FPDF**.
- The entire application is **containerized using Docker** and deployed on AWS EC2 through a **CI/CD pipeline (GitHub Actions)**

2.2. Process Flow

The system follows a structured execution flow:

- User sends request (e.g., "Create construction plan")
- Backend processes input
- GenAI module generates intelligent suggestions
- Backend validates and structures output
- Response returned to user



2.3. Information Flow

The flow of information in the system is as follows:

User → Frontend → Backend → GenAI Module → Backend → Frontend → User

2.4. Components Design

- **Component 1 – Docker file**

Property	Value
Base Image	python:3.11-slim
Build Type	Multi-stage build
Working Directory	/app
Exposed Port	8000
Entry Point	python app.py

- **Component 2 – Docker Compose**

Service	Description
app	Main construction service
db	Database(PostgreSQL/MySQL)
cache	Redis (optional)

- **Component 3 – CI/CD Pipeline**

(using GitHub Actions / Jenkins)

Step	Action
1	Checkout code
2	Install dependencies
3	Build Docker image
4	Run tests
5	Push image to Docker Hub
6	Deploy using Docker Compose

- **Component 4 – Docker hub Registry**

Action	Purpose
Push Image	Store container image
Pull Image	Used during deployment

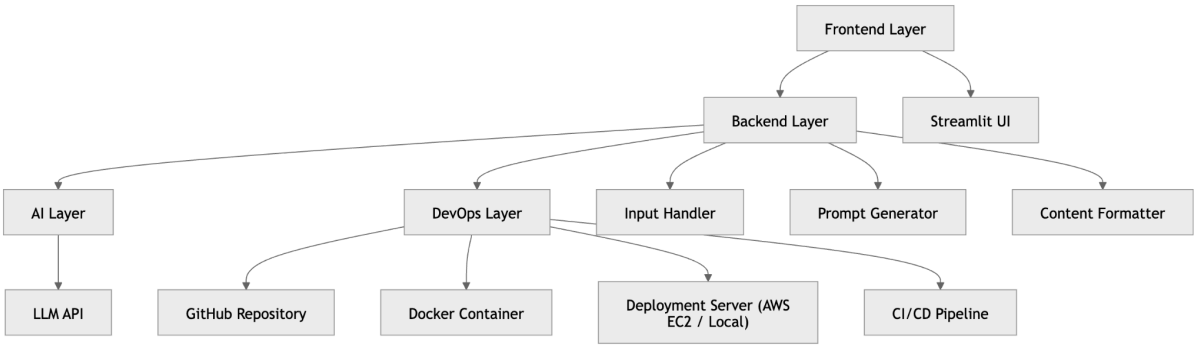


Fig 2.4 Component Diagram

2.5. Key Design Considerations

1. Multi-stage Docker build

- Reduces image size
- Improves security
- Removes unnecessary files

2. Separation of Concerns

- Application code and DevOps configs separated
- Enables parallel development

3.Environment Consistency

- Same container runs in dev, test, prod
- Eliminates "works on my machine" problem

4.GenAI Integration

- AI assists in construction planning
- Keeps logic modular and scalable

5.Security

- Minimal base images
- No secrets stored in image
- Environment variables used

6. Cost Efficiency

- Lightweight containers
- Minimal infrastructure usage

2.6. API Catalogue

1. Internal APIs

Endpoint	Method	Purpose
/generate-report	POST	Generate construction report using AI
/process-prompt	POST	Prepare input prompt for LLM
/llm-response	POST	Get response from Gemini LLM
/generate-pdf	POST	Convert report into PDF
/download-report	GET	Download generated report
/load-data	GET	Load knowledge base documents

External APIs

1. Google Gemini API

API	Method	Purpose
Gemini Generate Content API	POST	Generates construction reports based on user input and prompts

Authentication:

- API key via environment variables

3. Interfaces

The system provides both User Interface and External Interfaces for smooth interaction.

1. User Interface

- Built using Streamlit
- Allows users to:
    - Enter construction site details
    - Generate AI-based reports
    - View and download PDF reports

2. External Interfaces

- Google Gemini API
  Used for generating AI-based construction content
- File System (Data Folder)
  Stores knowledge base documents for retrieval

3. System Interfaces

- Vector Database (TF-IDF)
  Retrieves relevant documents for context-aware generation
- PDF Generator (FPDF)
  Converts generated content into downloadable reports

4. Data Design

The Data Design of the Construction GenAI system focuses on handling user inputs and AI-generated outputs efficiently without relying on persistent storage. The system follows a stateless, API-driven architecture, where data is processed in real-time and returned to the user. The design ensures simplicity, fast processing, and secure handling of API interactions.

4.1 Data Model

The system uses a **dynamic data model**, where data is generated and processed during runtime rather than stored permanently.

Key Data Entities:

- **User Input**
    - Construction query (e.g., planning, estimation, safety)
    - Format: String

- **Processed Prompt**
  - Refined input sent to LLM
  - Includes structured instructions

4.2 Data Access Mechanism

The system accesses and transfers data through API-based communication and DevOps pipeline flow.

Data Flow:

- User → Streamlit UI → Backend → LLM API → Response → UI

Access Methods:

- **HTTP Requests**
  - Used for communication with LLM APIs
- **GitHub**
  - Stores application code
- **Docker**
  - Packages application
- **Cloud (AWS EC2 / similar)**
  - Hosts application

4.3 Data Retention Policies

The system follows a no persistent storage policy for simplicity and security.

Policies:

- No database storage implemented
- Data exists only during user session
- No user history tracking
- No sensitive data stored

4.4 Data Migration

Currently, **data migration is not required** due to the absence of a database.

Current State:

- Stateless system
- No stored data
- No versioned data migration

5. State and Session Management

- Managed using Streamlit session state
- Each user session:
  - Is isolated
  - Stores temporary inputs and outputs
- No persistent storage is used
- Execution results are:
  - Generated dynamically
  - Lost after session ends or refresh

6. Caching

- No caching mechanism implemented in current system
- Each request:
  - Triggers fresh LLM execution

7. Functional Requirements

The system must:

- Accept construction site details from user
- Process input and generate AI prompt
- Retrieve relevant data from knowledge base
- Generate report using AI (Gemini)
- Convert report into PDF
- Display report to user
- Allow download of PDF report

7.1 User Authentication & Authorization

- No authentication system implemented
- Users access application directly
- API keys (Gemini/Groq):
  - Entered at runtime via UI
  - Not stored in code or database

7.2 Infrastructure Configuration Management

- Configuration managed via:
  - Environment variables
  - .env files (if used locally)
- Key configurations:
  - API keys
  - Model parameters

- System follows:
  - **Code-based configuration**
  - Easy portability across environments

7.3 Provisioning Control & Monitoring

Provisioning:

- Application runs locally via:
  - streamlit run app.py

Monitoring:

- Basic monitoring via:
  - Console logs
  - Streamlit runtime logs

7.4 Dockerfile (Application Containerization)

Configuration:

Property	Value
Base Image	python:3.10/3.11
Working Directory	/app
Port	8501
Entry Point	streamlit run app.py

Key Steps:

- Install dependencies
- Copy project files
- Expose port 8501
- Run Streamlit server

7.5 GitHub Actions (Workflow Configuration)

Workflow Steps:

1. Checkout repository
2. Install dependencies
3. Run tests (if any)
4. Build Docker image
5. Push image to registry
6. Deploy (optional)

Secrets Used:

- API keys
- Docker credentials

7.6 CI/CD Strategy (Continuous Integration & Deployment)

Continuous Integration:

- Trigger: git push
- Actions:
  - Code validation
  - Build process
  - Dependency installation

Continuous Deployment (Optional Extension):

- Deploy to:
  - Docker container
  - Cloud (AWS)

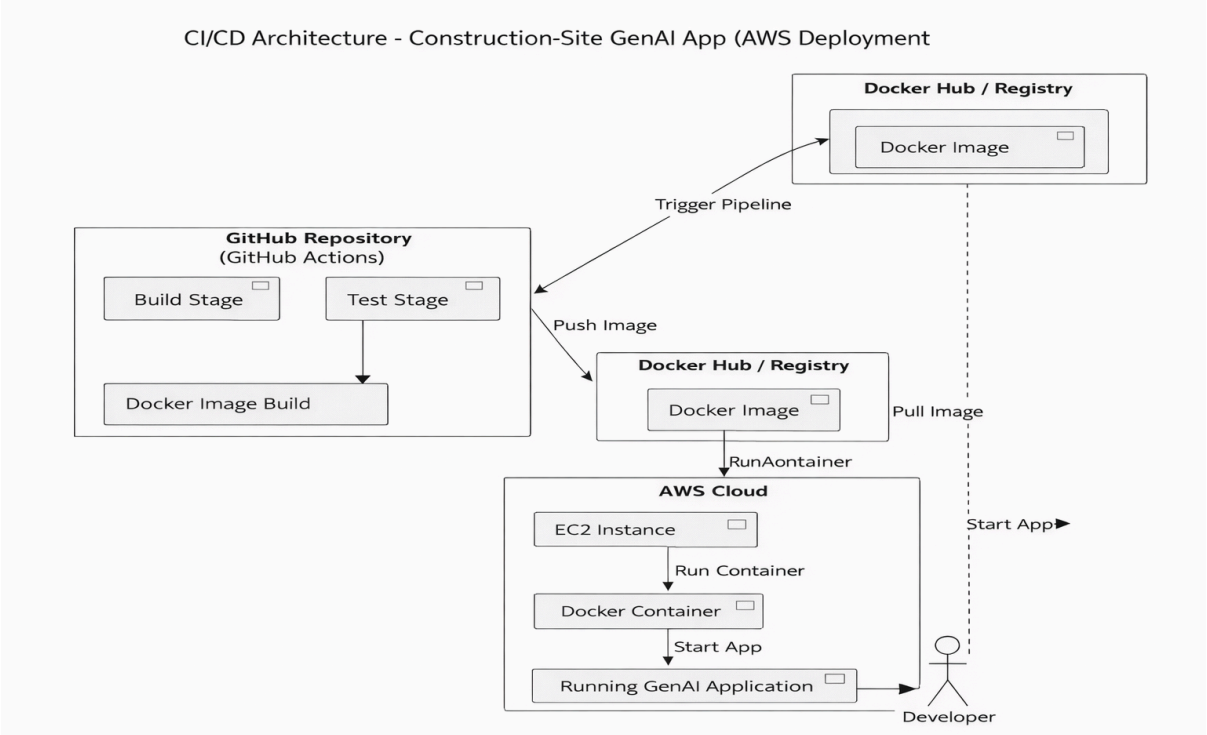


Figure 1: CI/CD Architecture Diagram

8. Non-Functional Requirements

- **Performance**: The system should generate construction plans within a reasonable response time, primarily dependent on LLM API latency.
- **Usability:** The user interface should be simple and intuitive, allowing users to input goals and view results without technical complexity.
- **Reliability:**The system should process each request independently to ensure consistent and predictable outputs without affecting other sessions.

9. References

- Docker Documentation
- GitHub Actions / Jenkins Docs
- GenAI API Documentation

10. GitHub

Github link - https://github.com/anshu2426/construction-genai